



COS30018 - Intelligent System

Project: Stock Price Prediction System

Task C.3 - Data Processing - Part 2

Student Name: Ngo Sy Vuong

Student ID: 105551480

Tutor: Nguyen Manh Toan

Tutorial Session: Wednesday 8:00 - 12:00

Semester: May - July 2026

Table of Contents

Content	Page
Cover Page	1
Table of Contents	2
Environment Setup	
1. Environment	3
2. Dependencies	3
3. Deployment	3
Executive Summary	4
Stock Prediction Version 3	
1 - Candlestick Chart	4
2 - Boxplot Chart	6

Environment Setup

To successfully replicate and run the stock price prediction model, please fulfill the following structural and software dependencies.

1. Environment

The system is built on an isolated virtual environment (venv) to ensure dependency abstraction and eliminate version conflicts with global system libraries:

- Core Runtime: **Python 3.11.x (64-bit)**
- Execution Interface: **Windows Command Prompt (CMD)**

2. Dependencies

tensorflow	Core Deep Learning framework used to compile structure and execute the LSTM network
yfinance	Automated financial data retrieval tool used to fetch historical structural data for CBA.AX via the Yahoo Finance API
scikit-learn	Provides data preprocessing mechanics
pandas	Sequential series concatenation, index cleaning and CSV file extraction
numpy	Low-level vectorized array manipulation, matrix dimension reshaping and numeric tensor optimization
matplotlib	Render visual evaluations
mplfinance	Specialized financial visualization extension built on Matplotlib
seaborn	Statistical data visualization framework built on top of Matplotlib

3. Deployment

To reconstruct this environment layout from scratch, run the following sequential commands within the target file directory: **(make sure you have installed Python 3.11)**

```
# Initialize the Python 3.11 Isolation layer
py -3.11 -m venv venv
```

```
# Activate the localized scripts block (Windows)
venv\Scripts\activate.bat
```

```
# Synchronize deployment prerequisites
pip install -r requirements.txt
```

Executive Summary

Before working on task C.3, the structure of the project has been improved. The task C.2 and C.3 inside `stock_prediction.py` will now be separated into `data_preprocessing.py` and `visualization.py`, respectively. This change makes the `stock_prediction.py` file cleaner and more readable.

Task C.3 adds 2 visualization tools into the project inside `visualization.py`. These functions are designed to look at the stock data under a microscope before training the model.

The visualization engines interact exclusively with raw, unscaled asset frames. This structural choice guarantees that all visualization coordinates map directly to nominal currency values (like \$100.00) rather than normalized scalar decimals between 0.0 and 1.0.

1 - Candlestick Chart

At the very beginning of the function, the use of `plot_df = dataframecopy()` creates an isolated, independent duplicate of the stock data in a separate memory location. This is an important defensive programming step because the subsequent downsampling and resampling operations required for the charts structurally alter row timelines and indices. By modifying a deep copy instead of the original reference, the script prevents these visual data transformations from corrupting the core dataset needed downstream to train the LSTM neural network.

```
# Using copy() to generate a deep copy of the dataframe in an isolated memory location.
# Reason: If using the original dataframe, the downstream calculation mutate the row indices,
#         which would cause structural array error when the data reaches the LSTM model.
plot_df = dataframe.copy()
```

Figure 1.1 - Create an isolated copy of the stock data before working with the chart

The function `display_candlestick_chart` displays a standard candlestick chart to show a stock's movement over a single day. However, the function has an aggregation parameter (`n`) that allows compressing multiple trading days into a single candle.

```
# Using resample(f'{n}B') to group the rows by specific time frequency.
# 'B' stands for Business days
# Using 'B' instead of 'D' (Calendar days) to ignore weekends and market holidays,
# eliminating empty rows
plot_df = plot_df.resample(f'{n}B').agg(aggregation_rules).dropna()
```

Figure 1.1 - Using aggregation parameter (`n`), frequency modifier ('B') and `aggregation_rules`

In financial context, averaging prices flattens out the data, completely erasing the extreme price spikes and drops that dictate market risk. To preserve this underlying financial truth, the function uses `aggregation_rules`. For example, if we want 5 trading days ($n = 5$) to equal 1 candle, the code will use a custom lookup map called `aggregation_rules` to combine those 5 days:

- **Open ('first')**: Takes the opening price from the very first day
- **High ('max')**: Scans all 5 days and takes the absolute highest peak reached
- **Low ('min')**: Scans all 5 days and takes the absolute lowest bottom dropped to
- **Close ('last')**: Takes the final closing price from the very last day
- **Volume ('sum')**: Adds all the shares traded over those 5 days together

```
# Define strict financial aggregation rules to preserve OHLC properties
aggregation_rules = {
    'Open': 'first',      # First opening price in the n-day window
    'High': 'max',        # Absolute highest peak across all n days
    'Low': 'min',         # Absolute lowest bottom across all n days
    'Close': 'last',     # Final closing price of the n-day window
    'Volume': 'sum'      # Total accumulated trading volume over the period
}
```

Figure 1.2 - Defining `aggregation_rules`

Furthermore, standard calendar grouping parameters naturally create empty data fragments containing NaN markers over weekends and market holidays, resulting in broken, glitchy visual lines. This function cleanly bypasses the weekend gap bug by anchoring its downsampling buckets to the specialized 'B' (Business day) frequency modifier, instructing Pandas to jump seamlessly from Monday to Friday. (See Figure 1.1)

Powered by the `mplfinance` framework, the function then produces a green and red theme (using 'charles' theme), while the lower panel charts volume bars along the exact same synchronized timeline index.

```
mpf.plot(
    plot_df,
    type='candle', # Geometric candlestick bodies and wicks
    style='charles', # standard green and red financial color theme
    title=f"\n{ticker} - {n}-Day Aggregated Candlestick Chart",
    ylabel='Stock Price ($)',
    ylabel_lower='Shares Traded',
    volume=True, # The lower panel displaying volume bars
)
```

Figure 1.3 - Using 'charles' theme and displaying volume bars

Candlestick chart using the train_df data:



Figure 1.4 - Candlestick Chart

2 - Boxplot Chart

While candlestick charts trace explicit pricing boundaries, they do not show statistical dispersion, price density or historical volatility shifts over time. The rolling boxplot addresses this limitation by partitioning target closing prices into independent, continuous chronological blocks and mapping their statistical characteristics without assuming a normal distribution.

Each individual box plot breaks down a localized chunk of numbers into a clear visual hierarchy. Seaborn automatically loops through each chunk in the window_data list to calculate and display the median baseline price as a solid center line, the core 50% trading zone as the Interquartile Range (IQR) box, the normal boundaries of market behavior as outer extending whiskers and wild, erratic price spikes as isolated outlier dots.

Slicing dense historical data day-by-day would cause severe layout crowding and overlapping text along the x-axis, making the chart impossible to read. To maintain a clean visual balance and a highly scannable canvas, the function calculates a programmatic step size called a stride, which is determined by taking half the lookback window size.

```

# Extract the raw data of the 'close' column
raw_prices = dataframe[target_col].values

# The absolute integer size of the dataset array
total_days = len(raw_prices)

window_data = []
window_labels = []

# Instead of sliding forward by 1 single day (which generates too many overlapping plots),
# it moves 10 days at a time to keep the chart well distributed
stride = max(1, window_size // 2)

```

Figure 2.1 - Vectorized Asset Extraction, Storage Initialization, and Downsampling Stride Configuration

For a standard 20 days lookup window, this results in a stride of 10 days, meaning the loop advances forward by 10 business days per iteration instead of sliding forward by a single day. At each step, the algorithm uses Python array slicing to extract a 20 days chunk of closing prices from the raw NumPy array (`raw_prices[start_idx:end_idx]`). Because Python slicing is exclusive of the upper bound, a slice like `[0:20]` plucks out exactly 20 elements from index 0 to 19 for the first window before appending it to the `window_data` matrix. (See the last line of Figure 2.1)

```

# The loop start at index 0
# Steps forward by the stride value
# total_days - window_size + 1 ensures that the loop stops before running out
# of rows at the end of the array
for start_idx in range(0, total_days - window_size + 1, stride):
    # The integer marker for the end of the current sliding window
    end_idx = start_idx + window_size

    # [start_idx:end_idx] is the Python array slicing syntax
    # Python slicing is exclusive of the upper bound
    # For example, if it is [0:20], then it will take from 0 to 19 for chunk 1
    chunk = raw_prices[start_idx:end_idx]
    # By the time the loop finishes, window_data becomes a list of arrays
    window_data.append(chunk)

    # Jump out of the raw NumPy numbers array and look back at the original Pandas dataframe.index
    # at the position matching start_idx
    # '%Y-%m-%d' stands for String Format Time, this strips away the hours, minutes and seconds
    # For example, 2026-06-06 12:00:00 will become 2026-06-06
    start_date = dataframe.index[start_idx].strftime('%Y-%m-%d')

    # Work exactly the same as start_date, but [end_idx - 1] is because index 19 is the 20th day
    end_date = dataframe.index[end_idx - 1].strftime('%Y-%m-%d')
    window_labels.append(f"{start_date} to {end_date}")

```

Figure 2.2 - Iterative Sliding Window Slicing and Chronological Label Boundary String Formatting

To prevent date bugs and align the text labels perfectly with the data, the function jumps out of the raw NumPy array and looks back at the original Pandas dataframe.index at the positions matching the boundary markers (start_idx and end_idx - 1). The minus 1 on the ending index is a critical boundary detail since index 19 represents the true 20th day of the slice, checking end_idx - 1 ensures the chart label points to the actual final day of the window rather than accidentally pulling the date of the 21st day. These 2 boundary timestamps are stripped of their hours, minutes and seconds using .strftime('%Y-%m-%d'), converting a raw value like 2026-06-06 12:00:00 into a clean string like '2026-06-06'. At the end of a loop, the start_date and end_date are combined into an explicit text range. (See the last 3 code line of Figure 2.2)

Finally, the function sets up a 14x7 inch Matplotlib canvas and formats the x-axis using plt.xticks(). This maps the integer layout positions across the horizontal plane to the descriptive date range strings, rotating the labels 45 degrees and aligning them to the right. (See Figure 2.3)

```
plt.figure(figsize=(14, 7)) # width = 14 inches, height = 7 inches

# data=window_data: This passes the 2D matrix list.
# Seaborn will loop through each chunk to calculate the median, upper/lower quartiles,
# whiskers and outlier dots for each window
sns.boxplot(data=window_data, color="#2ecc71")

plt.title(f"{ticker} Price Distribution Spread Over Rolling {window_size}-Day Windows", fontsize=14, fontweight='bold')
plt.xlabel("Sliding Temporal Windows (n Consecutive Trading Days)", fontsize=12)
plt.ylabel(f"Nominal '{target_col}' Price Valuation ($)", fontsize=12)

# Formats the text labels along the horizontal x-axis
# ticks=range(): Maps integer locations across the horizontal array plane
# labels=window_labels: Apply the (f"{start_date} to {end_date}") to those positions
plt.xticks(ticks=range(len(window_labels)), labels=window_labels, rotation=45, ha='right', fontsize=9)
plt.grid(True, linestyle=':', alpha=0.6)
plt.tight_layout()
plt.show()
```

Figure 2.3 - Seaborn Distribution Engine Mapping and Matplotlib Canvas Axis Label Optimization

Boxplot chart using the train_df data:

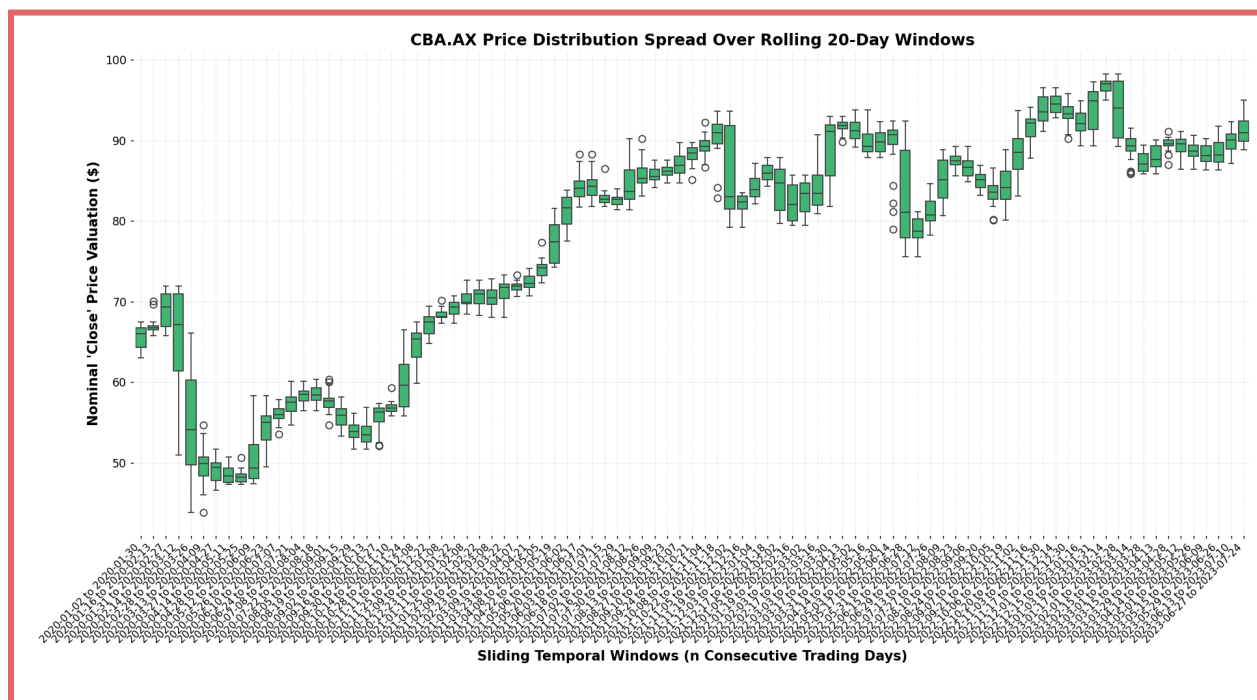


Figure 2.4 - Boxplot Chart